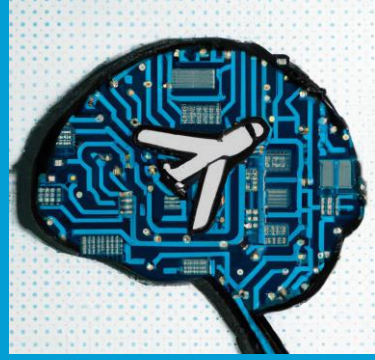
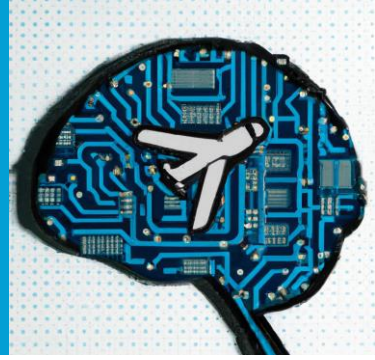




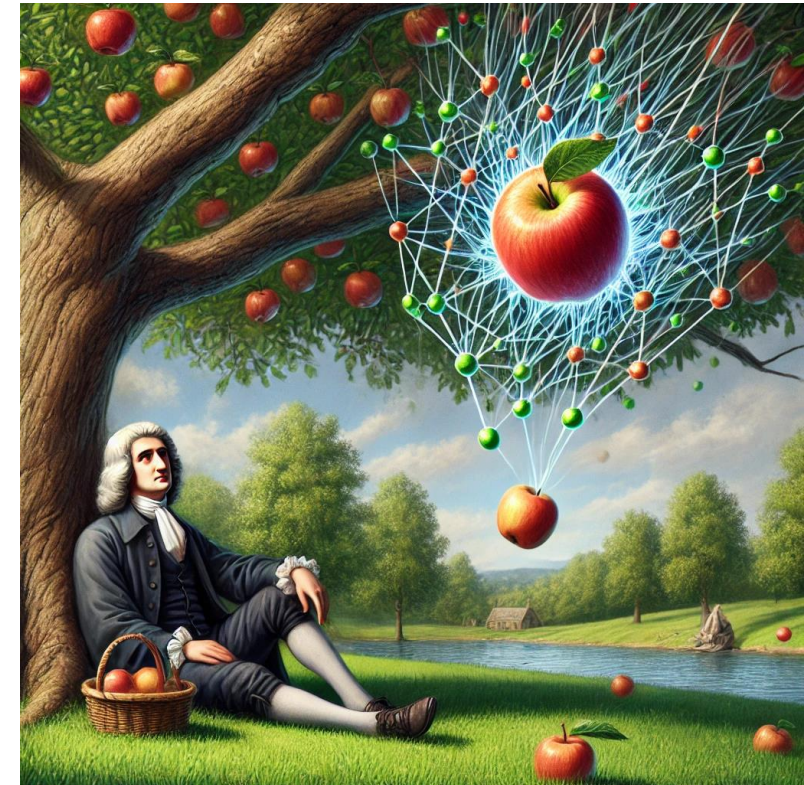
Physics-informed neural networks

Lecture objectives

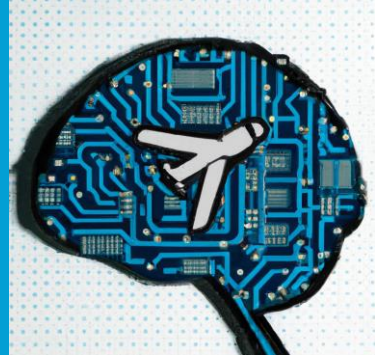


Students can:

- List the main approaches to design physics-informed neural networks (PINNs)
- Explain both the motivation behind PINNs and how they work.
- Apply PINNs to aerospace problems.

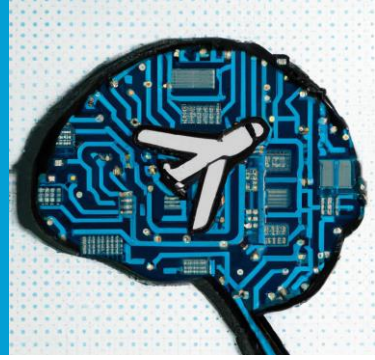


Physics-informed neural networks (PINNs)



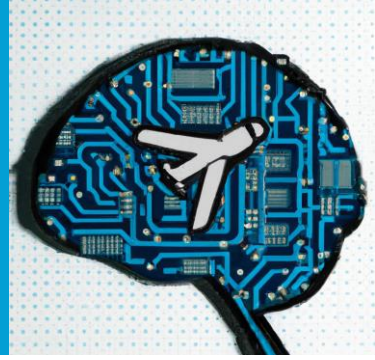
Leverage the high capacity of artificial neural networks for learning unknown, complex functions, while **adding guidance and constraints from physics** to regularize the learning.

Physics-informed neural networks (PINNs)



Why would you want to do this?

Physics-informed neural networks (PINNs)

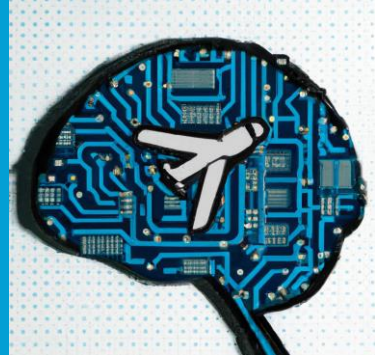


Why would you want to do this?

Envisaged benefits:

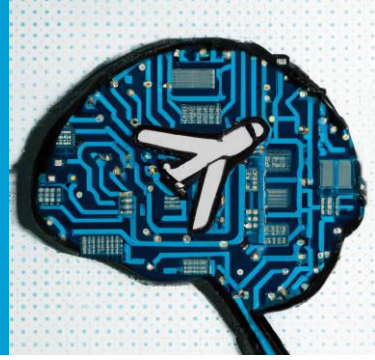
- ✓ Better inductive bias (and hence generalization to unseen cases)
- ✓ Quicker learning
- ✓ Solving complex problems with a short inference time
- ✓ Capture hard-to-model phenomena

Different types of PINNs



1. PINNs as the solution to Partial Differential Equations
2. PINNs as dynamical system
3. Other forms of PINNs

Example: 1D wave equation

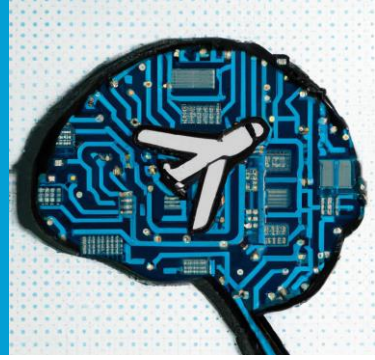


The wave equation is a second-order linear partial differential equation.

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$



Example: 1D wave equation



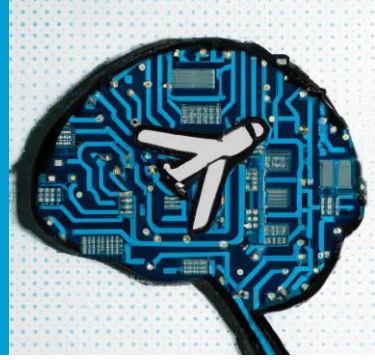
The wave equation is a second-order linear partial differential equation.

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

Would anyone know how to solve this?



Example: 1D wave equation



The wave equation is a second-order linear partial differential equation.

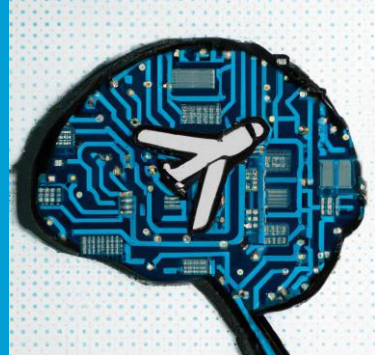
$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

Would anyone know how to solve this?

There are algebraic solutions for the infinite and finite case with boundary conditions.



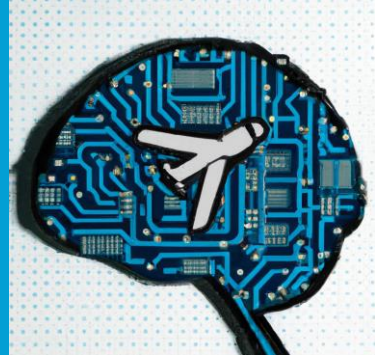
Example: 1D wave equation



1. Simulating the system with numerical integration.

How would you do it?

Example: 1D wave equation



1. Simulating the system with numerical integration.
One option: Transform the PDE to an ODE and use ODE solvers.

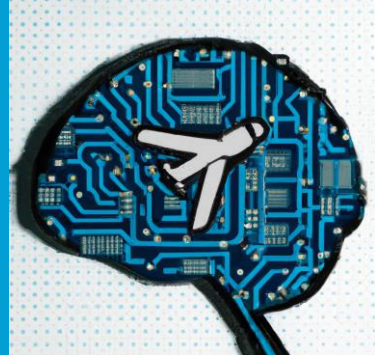
Introduce a variable $v = \frac{\partial u}{\partial t}$ leading to the ODE:

$$\frac{\partial u}{\partial t} = v$$

$$\frac{\partial v}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2}$$

, where $\frac{\partial^2 u}{\partial x^2}$ is approximated with finite differences.

Example: 1D wave equation



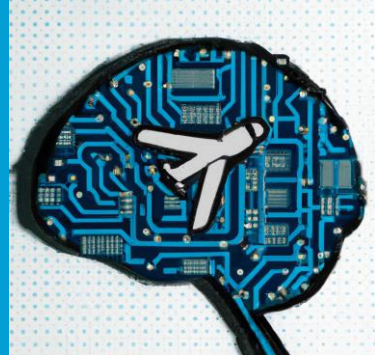
1. Simulating the system with numerical integration.

One option: Transform the PDE to an ODE and use ODE solvers.

```
# Define the system of first-order ODEs
def wave_ode(t, state):
    ...
    # Time derivatives
    du_dt = v # Velocity (u_t)
    dv_dt = c**2 * u_xx # Acceleration (v_t)

    return np.concatenate([du_dt, dv_dt])
    ...
# Solve the system of ODEs
solution = solve_ivp(wave_ode, [0, T], U0, t_eval=t_eval, method='RK45')
```

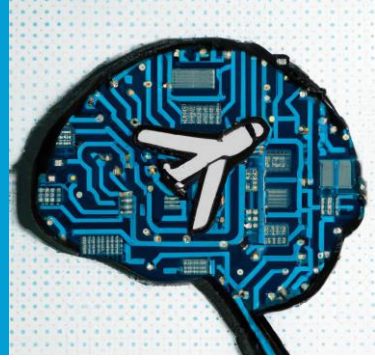
PINNs as the solution



Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

Free learning from data

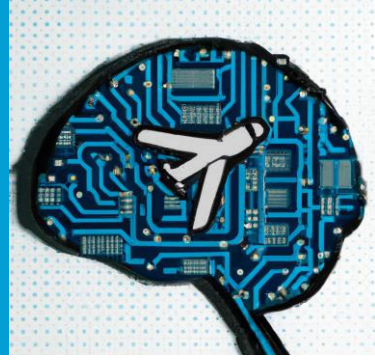


Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

We could learn this function directly by means of target samples u^* .

Free learning from data



Directly learn the solution with an ANN representing the function f :

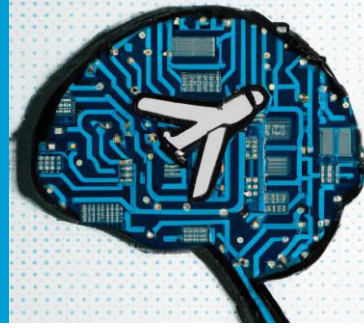
$$u(x, t) = f(x, t)$$

We could learn this function directly by means of target samples u^* .

However:

- Target samples may not be available.
- The network could overfit the data and generalize poorly.

Physics-informed learning

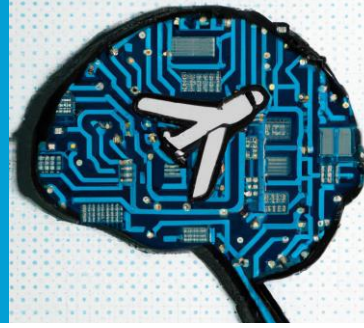


Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

Instead, we can use our knowledge of the physical system to regularize learning.

Physics-informed learning



Directly learn the solution with an ANN representing the function f :

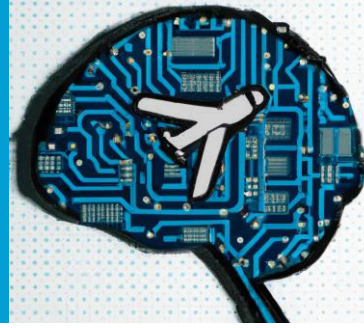
$$u(x, t) = f(x, t)$$

f has to obey:

The PDE residual

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

Physics-informed learning



Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

f has to obey:

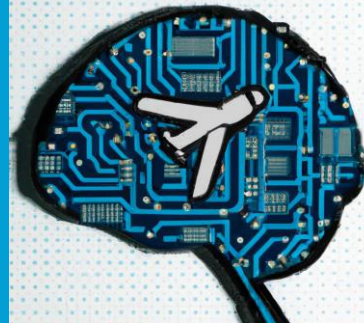
The PDE residual

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

Initial conditions

$$u(x, 0)$$

Physics-informed learning



Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

f has to obey:

The PDE residual

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

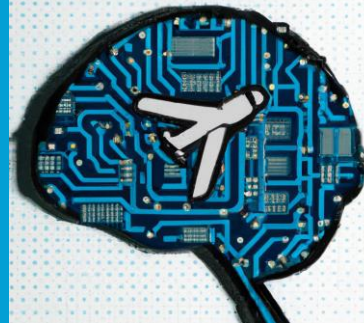
Initial conditions

$$u(x, 0)$$

Boundary conditions

$$\forall t: u(b_1, t) = c_1, \quad u(b_2, t) = c_2$$

Physics-informed learning



Directly learn the solution with an ANN representing the function f :

$$u(x, t) = f(x, t)$$

f has to obey:

The PDE residual

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

Initial conditions

$$u(x, 0)$$

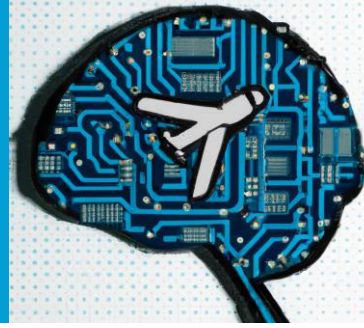
Boundary conditions

$$\forall t: u(b_1, t) = c_1, \quad u(b_2, t) = c_2$$

Available data points

$$u(x, t) = u^*(x, t)$$

Loss function with soft constraints

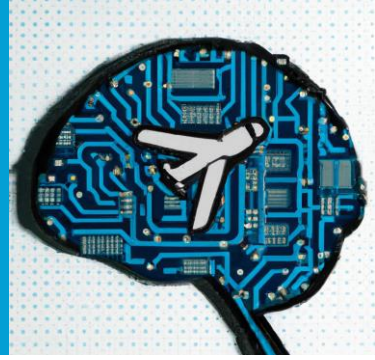


We can teach the network to respect all conditions in a “soft” manner by including all restrictions in the loss function:

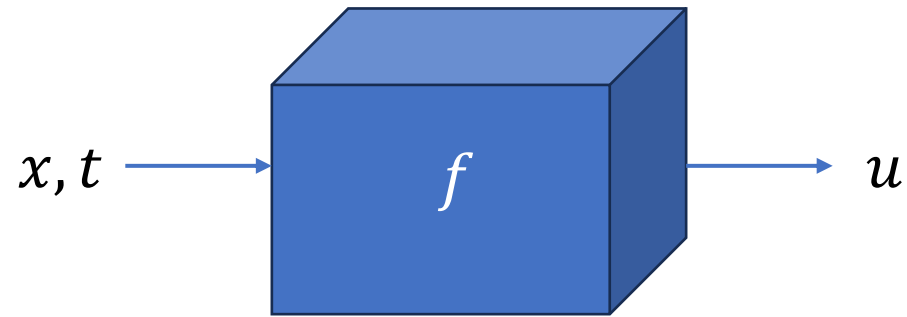
$$\mathcal{L} = \mathcal{L}_{\mathcal{F}} + \mathcal{L}_{\mathcal{B}} + \mathcal{L}_{\mathcal{I}} + \mathcal{L}_{\mathcal{D}}$$

Where $\mathcal{F}$ stands for the differential equations, $\mathcal{B}$ the boundary conditions, $\mathcal{I}$ the initial conditions, and $\mathcal{D}$ the data.

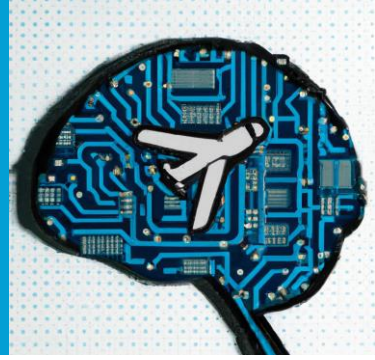
Determining $\mathcal{L}_{\mathcal{F}}$



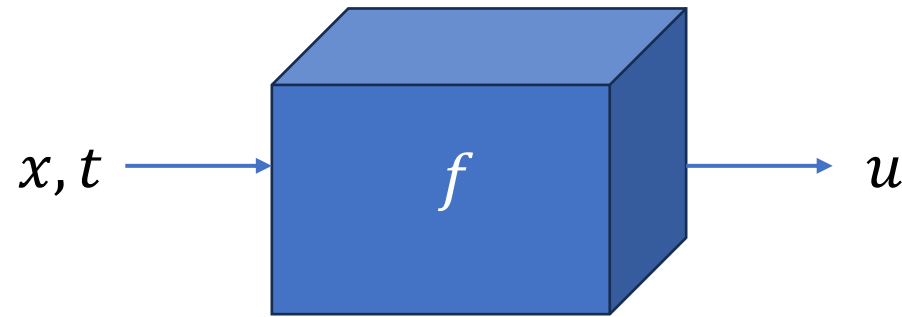
How would you determine $\mathcal{L}_{\mathcal{F}}$?



Determining $\mathcal{L}_{\mathcal{F}}$

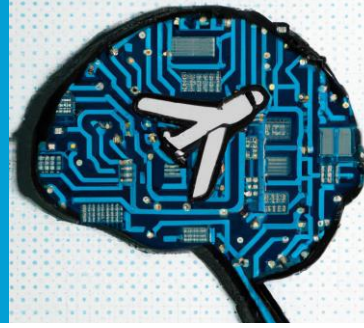


How would you determine $\mathcal{L}_{\mathcal{F}}$?

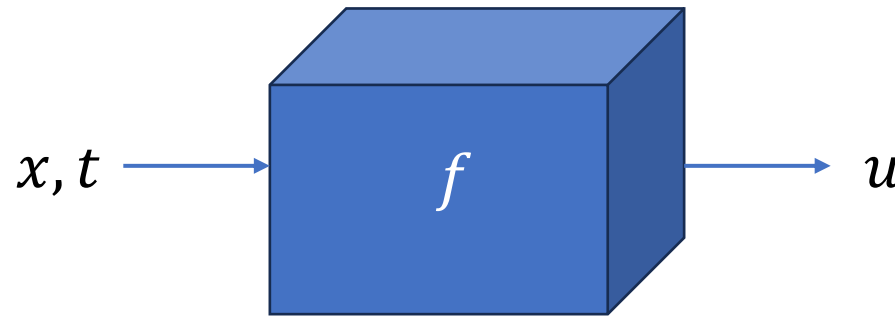


We can leverage automated differentiation! 💡

Determining $\mathcal{L}_{\mathcal{F}}$



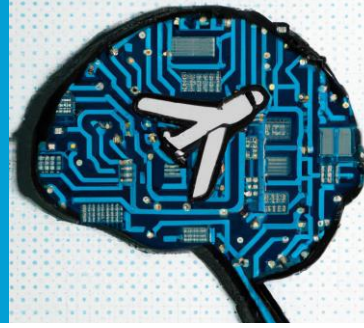
How would you determine $\mathcal{L}_{\mathcal{F}}$?



We can leverage automated differentiation! 💡

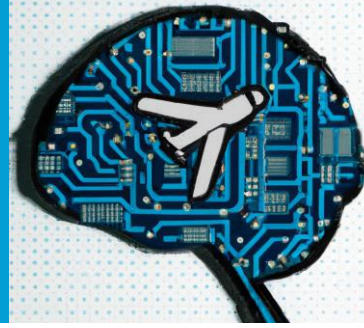
Using the chain rule as in backprop, it can determine $u_x = \frac{\partial u}{\partial x}$ and $u_t = \frac{\partial u}{\partial t}$, but also u_{xx} and u_{tt} . The results can be inserted into $\mathcal{L}_{\mathcal{F}}$.

Residual loss function in PyTorch



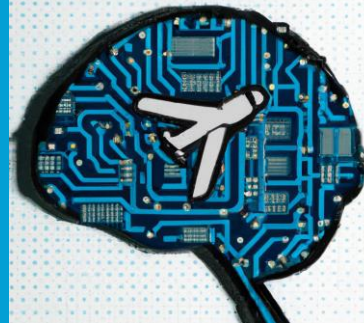
```
def wave_equation_residual(model, t, x, c):  
    u = model(t, x)  # Predicted u(t, x)  
  
    # Compute the derivatives:  
    u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_tt = torch.autograd.grad(u_t, t, grad_outputs=torch.ones_like(u_t),  
create_graph=True)[0]  
    u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),  
create_graph=True)[0]  
  
    residual = u_tt - c**2 * u_xx  
    return residual
```

Residual loss function in PyTorch



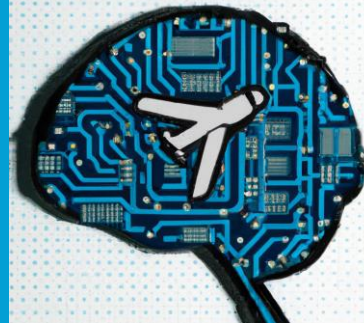
```
def wave_equation_residual(model, t, x, c):  
    u = model(t, x)  # Predicted u(t, x)  
  
    # Compute the derivatives:  
    u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_tt = torch.autograd.grad(u_t, t, grad_outputs=torch.ones_like(u_t),  
create_graph=True)[0]  
    u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),  
create_graph=True)[0]  
  
    residual = u_tt - c**2 * u_xx  
    return residual
```

Residual loss function in PyTorch



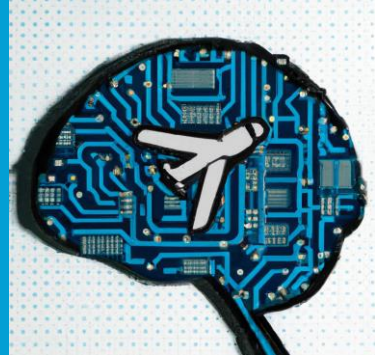
```
def wave_equation_residual(model, t, x, c):  
    u = model(t, x)  # Predicted u(t, x)  
  
    # Compute the derivatives:  
    u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_tt = torch.autograd.grad(u_t, t, grad_outputs=torch.ones_like(u_t),  
create_graph=True)[0]  
    u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),  
create_graph=True)[0]  
  
    residual = u_tt - c**2 * u_xx  
    return residual
```

Residual loss function in PyTorch



```
def wave_equation_residual(model, t, x, c):  
    u = model(t, x)  # Predicted u(t, x)  
  
    # Compute the derivatives:  
    u_t = torch.autograd.grad(u, t, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_tt = torch.autograd.grad(u_t, t, grad_outputs=torch.ones_like(u_t),  
create_graph=True)[0]  
    u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),  
create_graph=True)[0]  
    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),  
create_graph=True)[0]  
  
    residual = u_tt - c**2 * u_xx  
    return residual
```

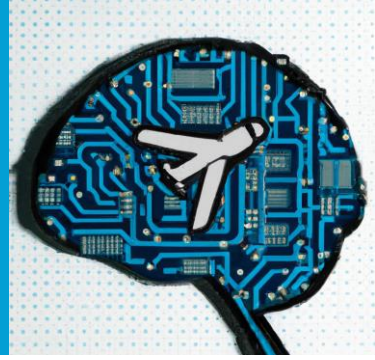
Different types of solutions



PINNs can be used for:

1. Forward problems
2. Inverse problems
3. Ill-posed problems
4. Generalization problems

Forward problems



PINNs can be used to solve forward problems, in which they solve for the outputs $\mathbf{u}$ given the state $\mathbf{x}$ and time t .

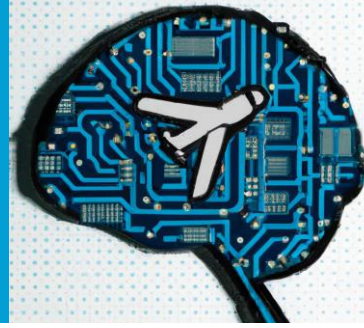
The network then represents f : $\mathbf{u} = f(\mathbf{x}, t)$

Subject to general form differential equations expressed as:

$$\begin{aligned}\mathcal{F}(\mathbf{u}(\mathbf{x})) &= g(\mathbf{x}), & \mathbf{x} \in \Omega \\ \mathcal{B}(\mathbf{u}(\mathbf{x})) &= h(\mathbf{x}), & \mathbf{x} \in \partial\Omega\end{aligned}$$

With Ω the state domain, and $\partial\Omega$ its border.

Forward problems



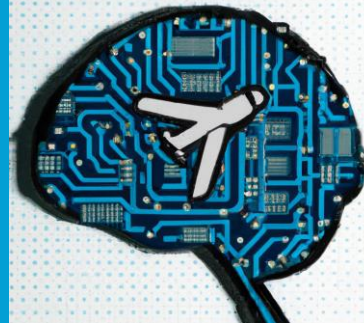
PINNs can be used to solve forward problems, in which they solve for the outputs $\mathbf{u}$ given the state $\mathbf{x}$ and time t .

The network then represents f :
$$\mathbf{u} = f(\mathbf{x}, t)$$

In **forward** problems, the **parameters** of the differential equations are considered to be **known**. We can also train them without data (no $\mathcal{L}_D$).

For the 1D wave equation c^2 is assumed to be known.

Inverse problems

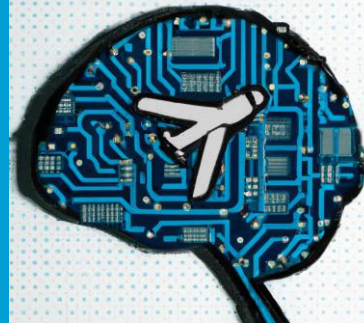


In **inverse** problems, we need to **optimize** also the **parameter values**.

In the 1D wave equation, we need to also find c^2 .

```
# Initialize speed with a learnable parameter
self.c = nn.Parameter(torch.tensor(initial_c, dtype=torch.float32))
```

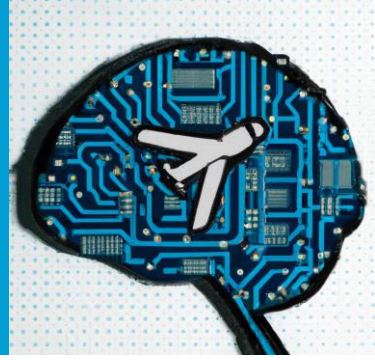
Ill-posed problems



Jacques Hadamard's conditions for a well-posed problem:

1. There problem has a solution.
2. The solution is unique.
3. The solution changes continuously based on changes in initial conditions.

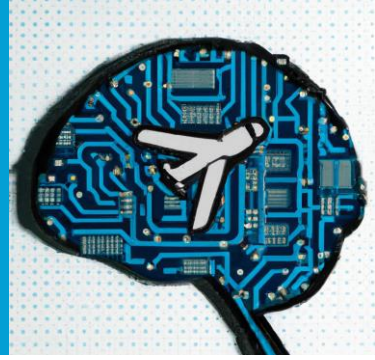
Ill-posed problems



Conditions that lead to ill-posed problems for PINNs:

1. The data might be insufficient or too noisy
2. There may be multiple possible solutions.
3. The modelled process may be (close to) chaotic.

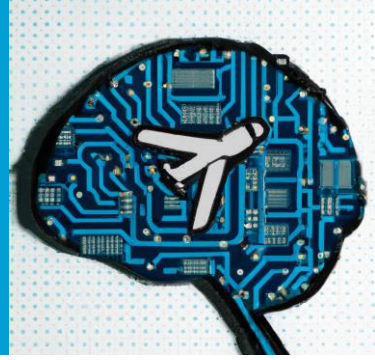
Ill-posed problems



Conditions that lead to ill-posed problems for PINNs:

1. The data might be insufficient or too noisy
2. There may be multiple possible solutions.
3. The modelled process may be (close to) chaotic.

Ill-posed problems

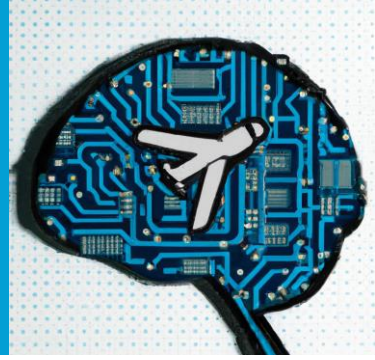


Conditions that lead to ill-posed problems for PINNs:

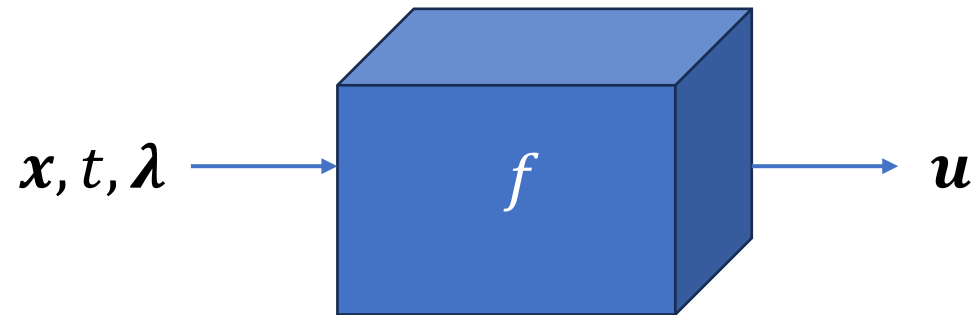
1. The data might be insufficient or too noisy
2. There may be multiple possible solutions.
3. The modelled process may be (close to) chaotic.

The network will generate a solution in any case, but generalization of the solution may be inadequate.

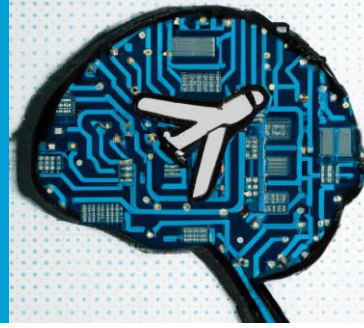
Generalization problems



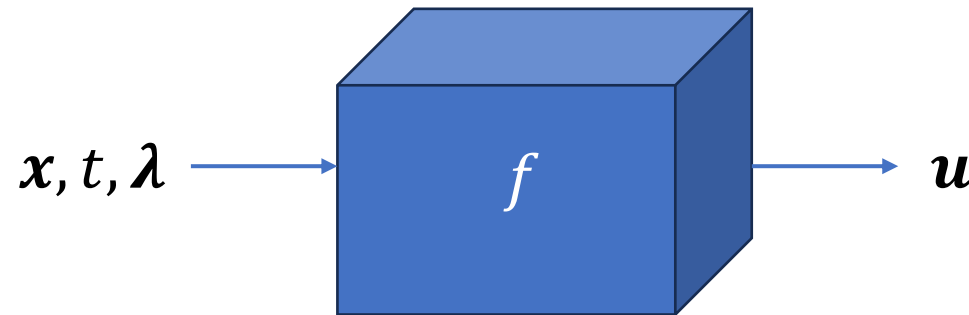
Have a PINN represent the solution to a whole range of systems, providing the network with the parameters λ .



Generalization problems

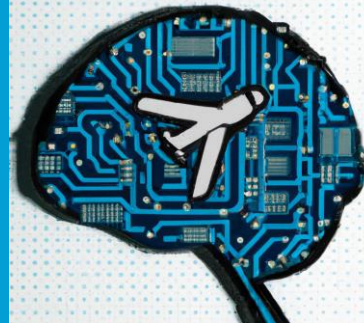


Have a PINN represent the solution to a whole range of systems, providing the network with the parameters λ .

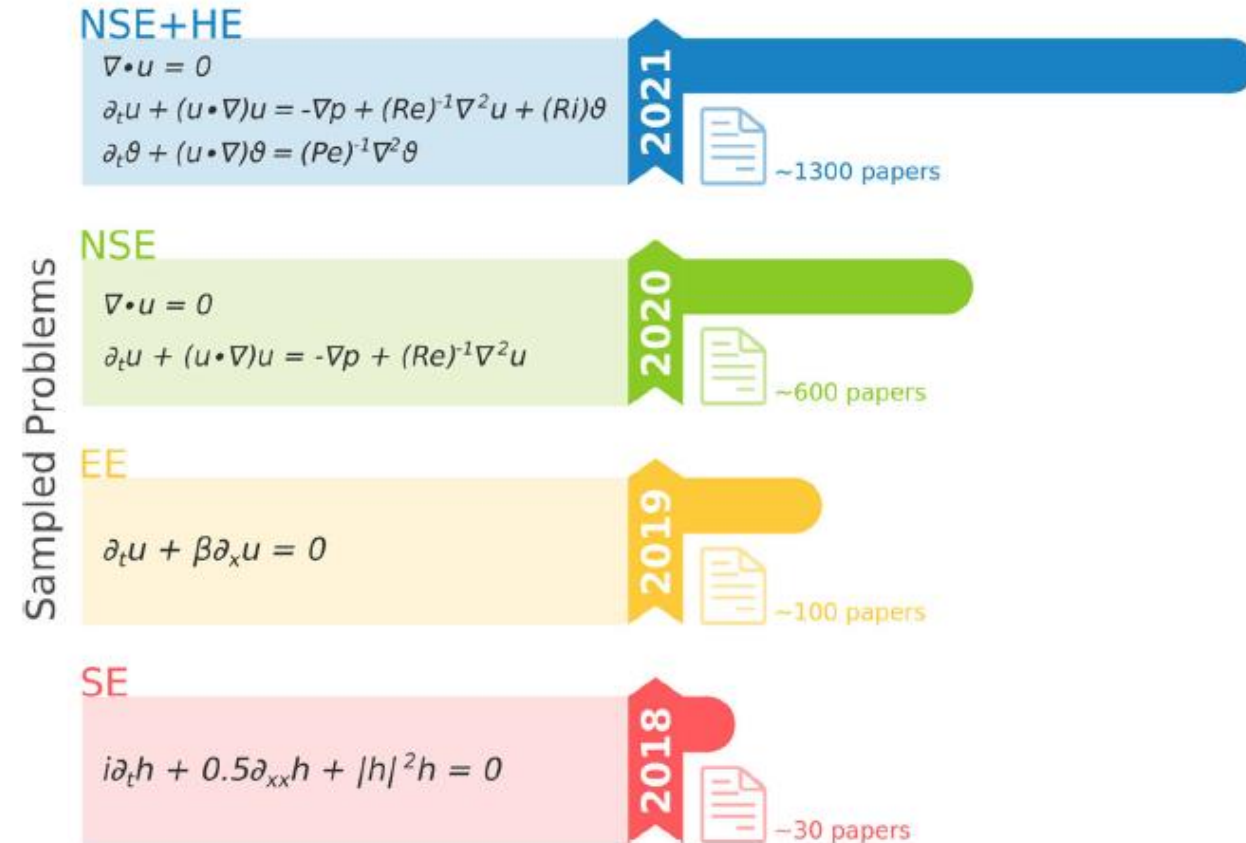


For the 1D wave equation, this means providing the speed c^2 as input, and training for different speeds.

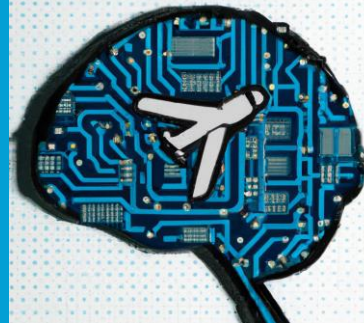
Challenges in learning PINNs



PINNs have been applied to increasingly complex problems.



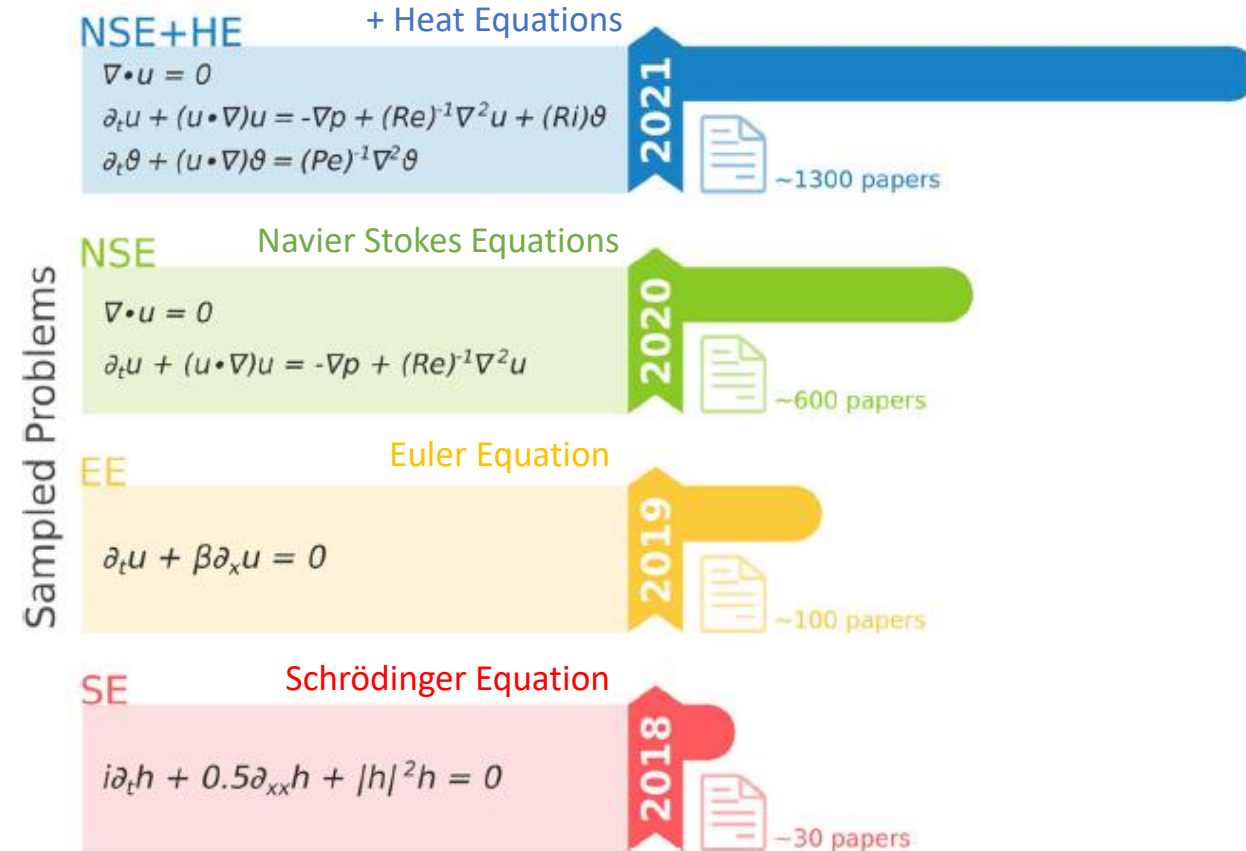
Challenges in learning PINNs



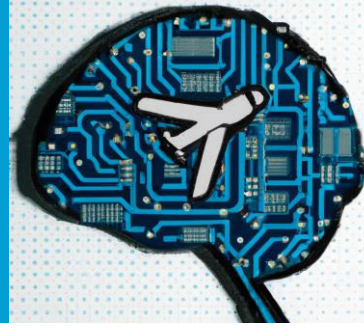
PINNs have been applied to increasingly complex problems.

PINNs still struggle with dynamical systems that are:

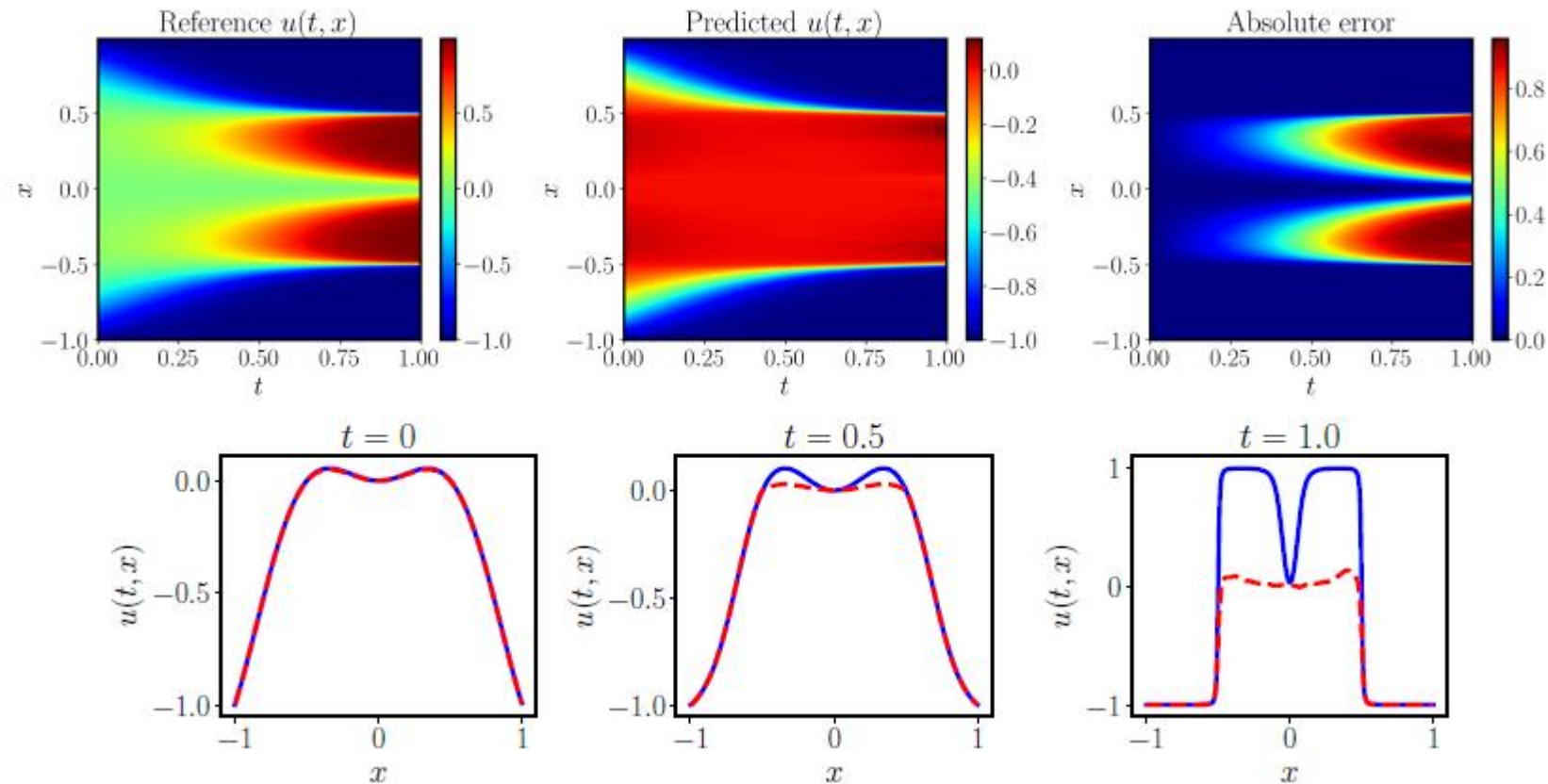
- Multi-scale
- Chaotic
- Turbulent



Vanilla PINNs do not respect causality

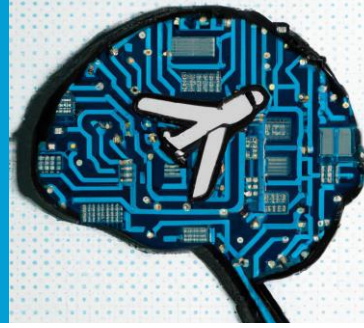


PINN applied to Allen-Cahn equations of reaction-diffusion:



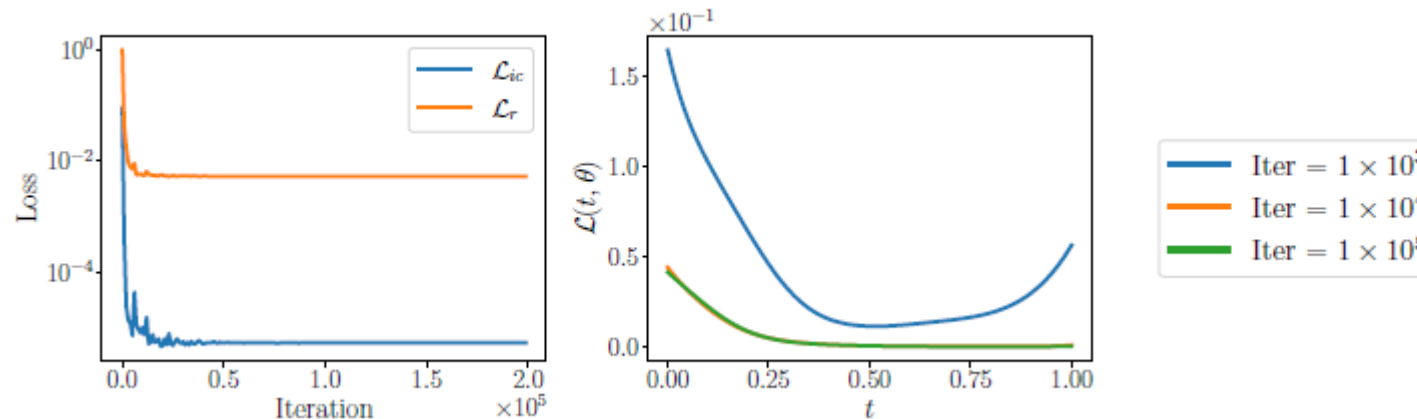
Wang, S., Sankaran, S., & Perdikaris, P. (2022). Respecting causality is all you need for training physics-informed neural networks. arXiv preprint arXiv:2203.07404.

Vanilla PINNs do not respect causality



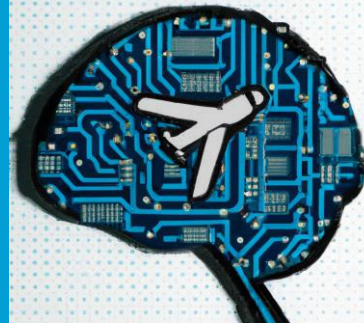
The PINN gets stuck in a local minimum:

- The loss curve suggests that training stagnates.
- The loss first goes down for the later time steps, and stays high for the initial conditions.



Wang, S., Sankaran, S., & Perdikaris, P. (2022). Respecting causality is all you need for training physics-informed neural networks. arXiv preprint arXiv:2203.07404.

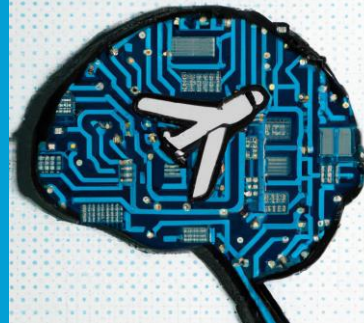
Modified loss function



Wang et al. propose a loss function that weighs the losses at different times t differently.

$$\mathcal{L}_{\mathcal{F}} = \frac{1}{T} \sum_{i=1}^T w_i \mathcal{L}_{\mathcal{F}}(t_i)$$

Modified loss function



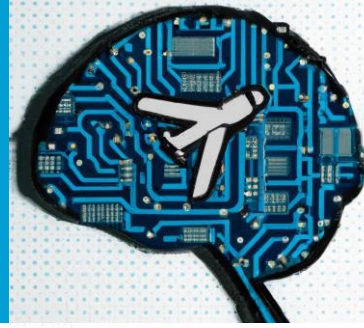
Wang et al. propose a loss function that weighs the losses at different times t differently.

$$\mathcal{L}_{\mathcal{F}} = \frac{1}{T} \sum_{i=1}^T w_i \mathcal{L}_{\mathcal{F}}(t_i)$$

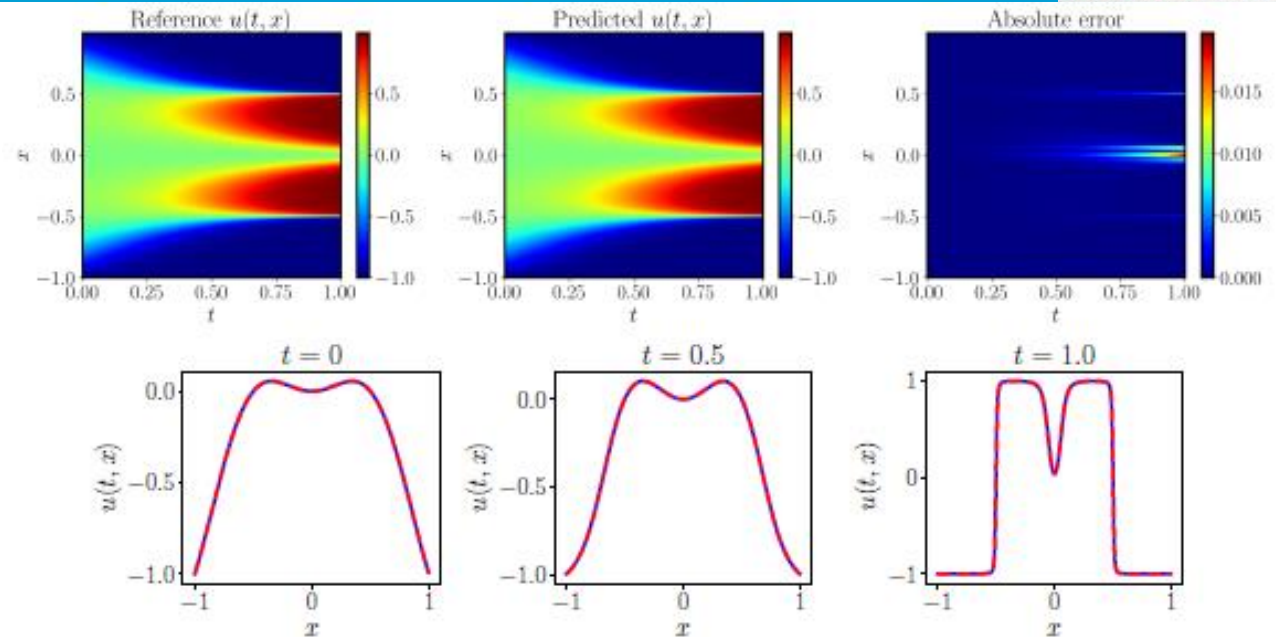
Losses at earlier time steps should receive a **larger weight** in the loss function, until the residual at these steps is sufficiently close to zero.

$$w_i = \exp\left(-\varepsilon \sum_{k=1}^{i-1} \mathcal{L}_{\mathcal{F}}(t_k) \right)$$

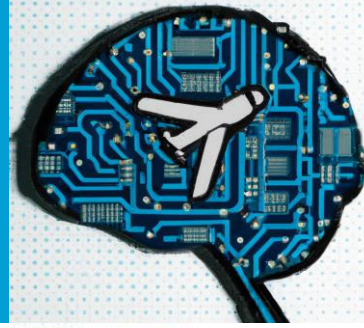
Results with the modified loss function



The results are better...

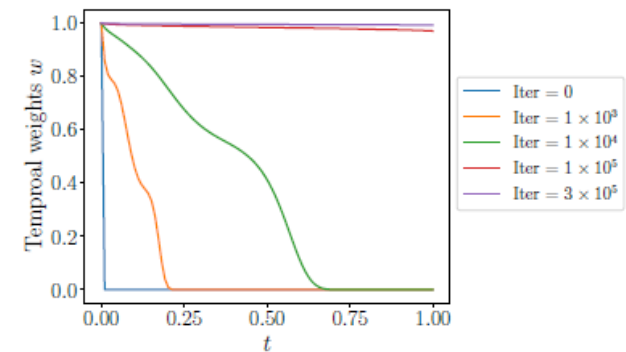
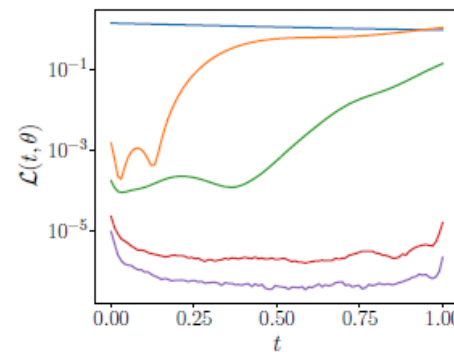
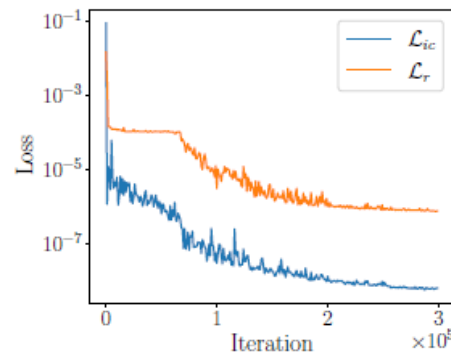
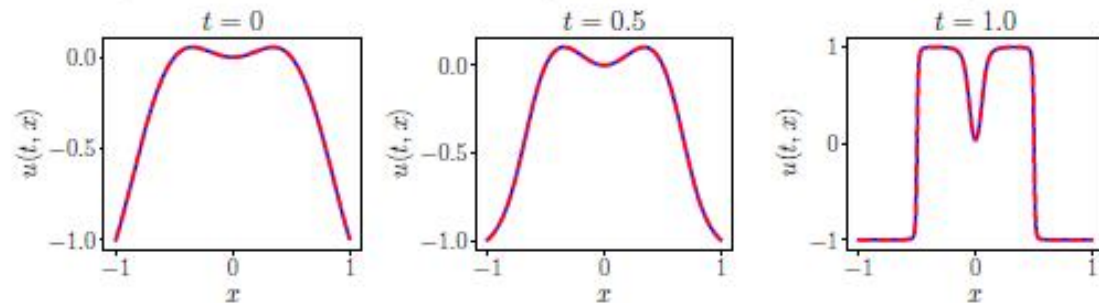
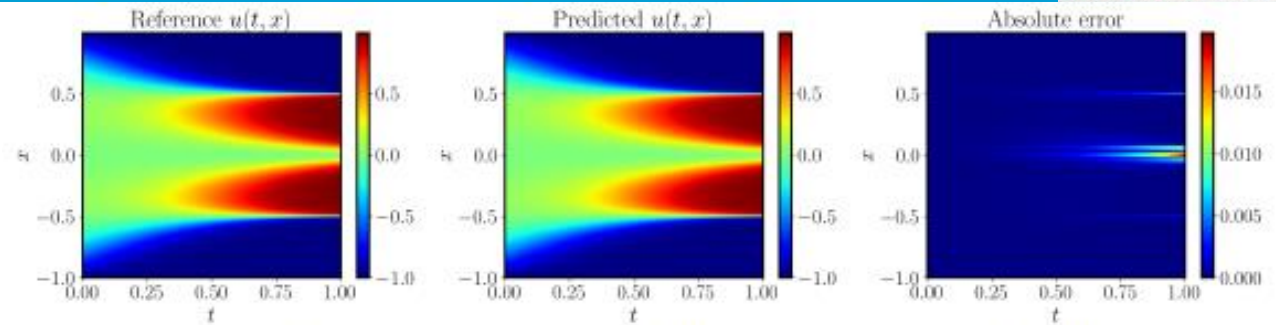


Results with the modified loss function

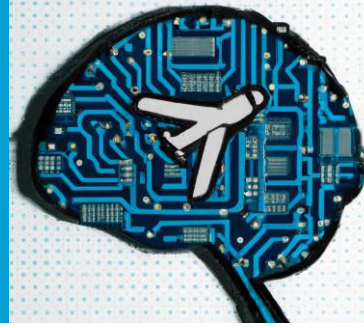


The results are better...

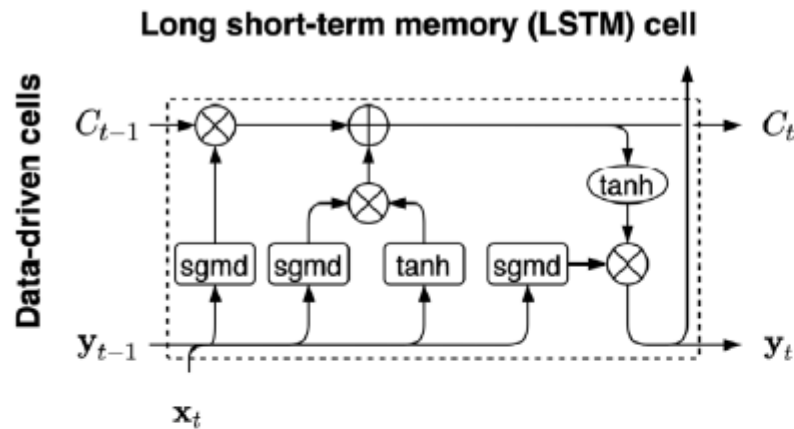
With lower error both at the start and end of the time domain.



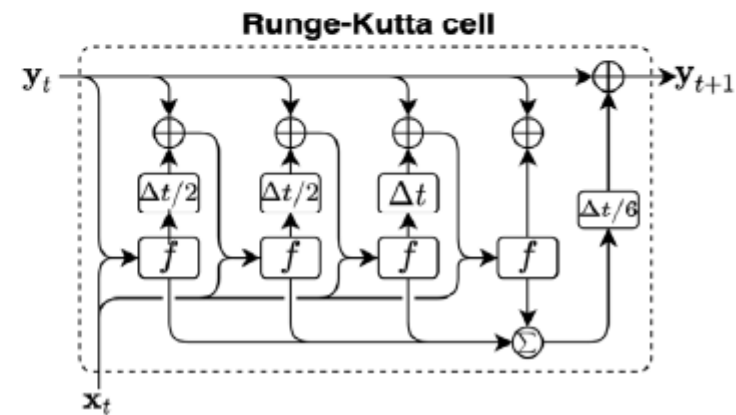
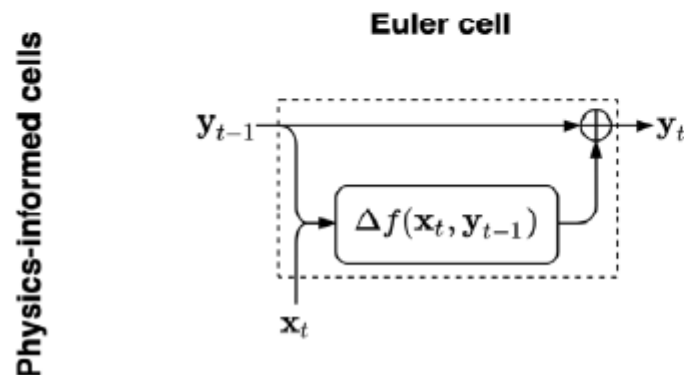
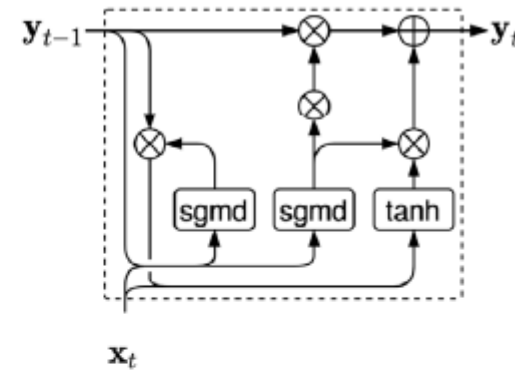
PINNs as dynamical systems



A PINN can also represent the dynamical system directly:

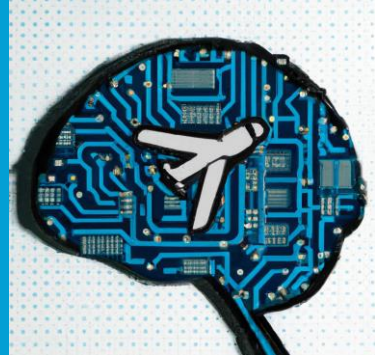


Gated recurrent unit (GRU) cell



Nascimento, Renato G., Kajetan Fricke, and Felipe AC Viana. "A tutorial on solving ordinary differential equations using Python and hybrid physics-informed neural network." Engineering Applications of Artificial Intelligence 96 (2020): 103996.

PINNs as dynamical systems



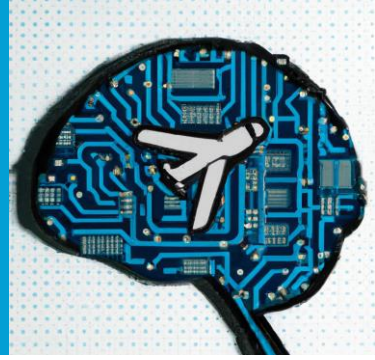
A PINN can also represent the dynamical system directly.

Basically, create a PyTorch module as a numerical integrator.

The module can include directly learnable parameters or use other neural network elements, such as MLPs, to estimate parameter dependence on the state.

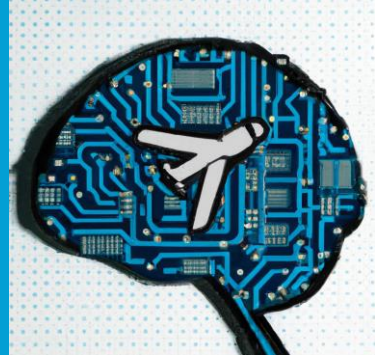
Nascimento, Renato G.,
Kajetan Fricke, and Felipe
AC Viana. "A tutorial on
solving ordinary differential
equations using Python and
hybrid physics-informed
neural network."
Engineering Applications of
Artificial Intelligence 96
(2020): 103996.

Other forms of PINNs



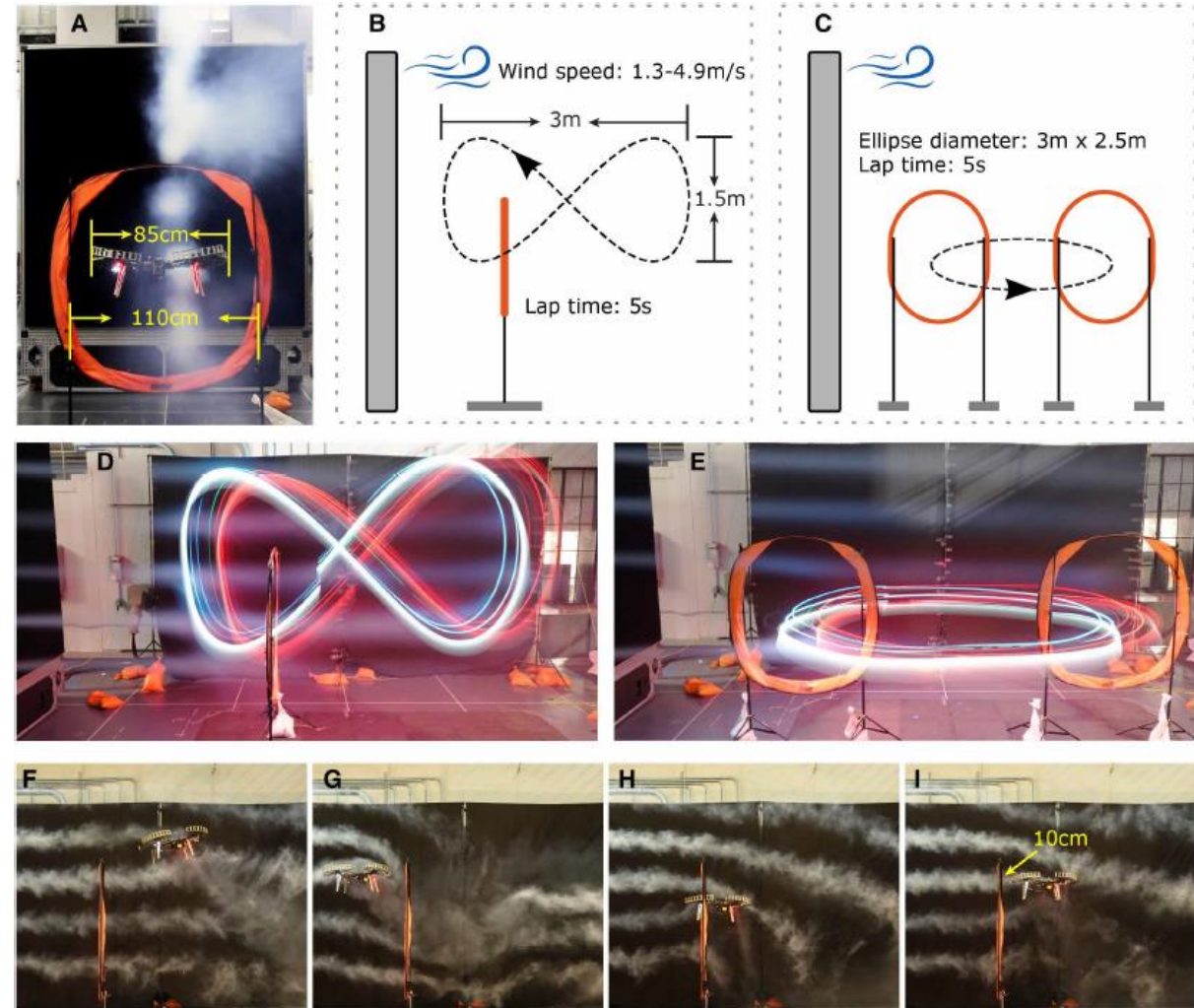
Also neural networks that estimate residuals are viewed as PINNs.

Neural-fly



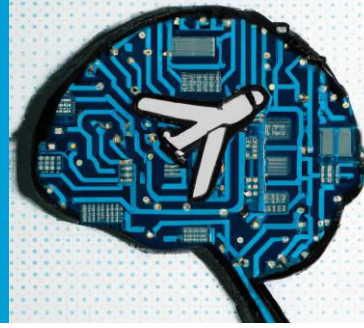
Example:

Neural network that learns basis functions for estimating aerodynamic force residuals.

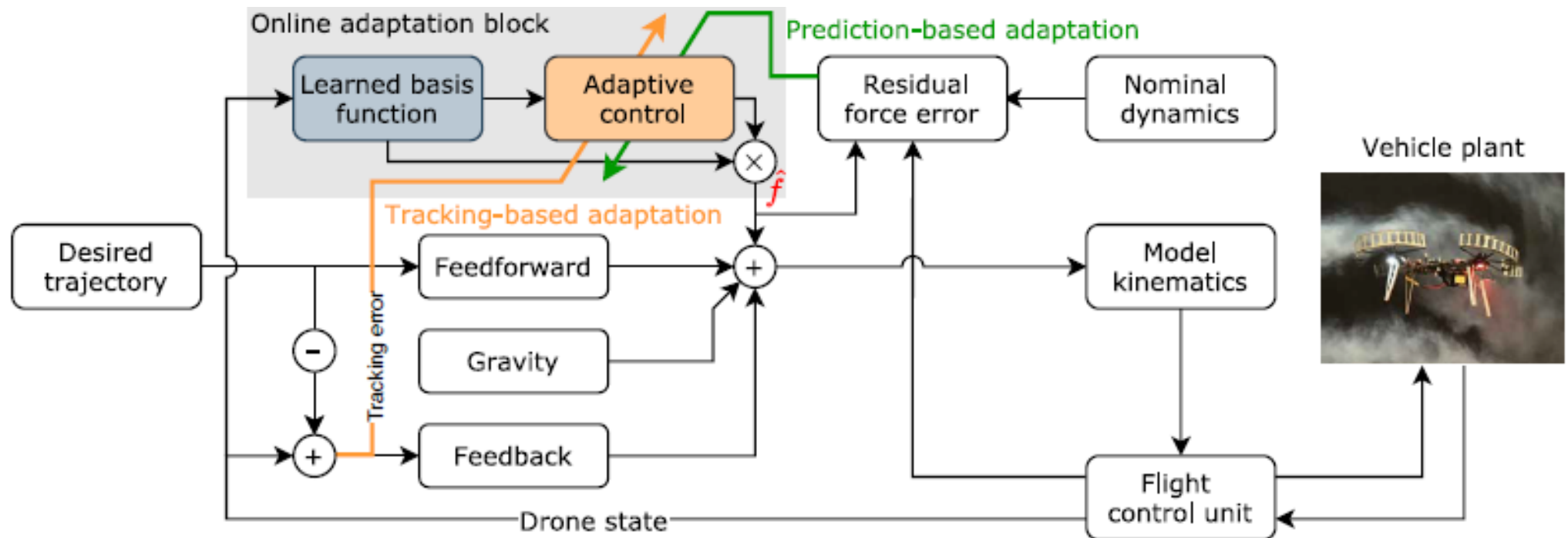


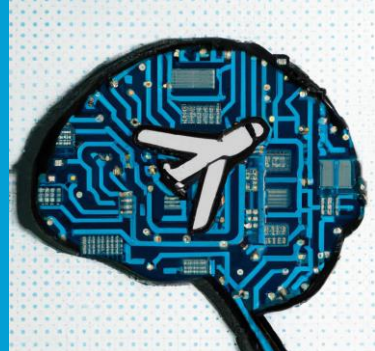
O'Connell, M., Shi, G., Shi, X., Azizzadenesheli, K., Anandkumar, A., Yue, Y., & Chung, S. J. (2022). Neural-fly enables rapid learning for agile flight in strong winds. *Science Robotics*, 7(66), eabm6597.

Neural-fly



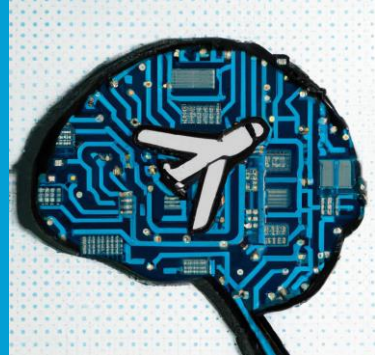
The neural network outputs a vector (modeling basis functions), which is element-wise multiplied with another vector for residual force prediction.





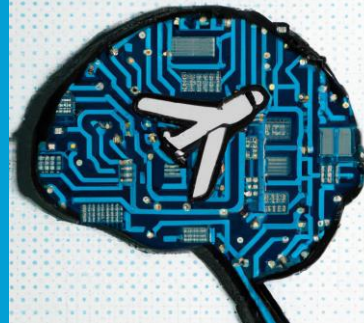
Quiz

Question 1:



Why do neural networks have problems with modeling the solution to chaotic systems?

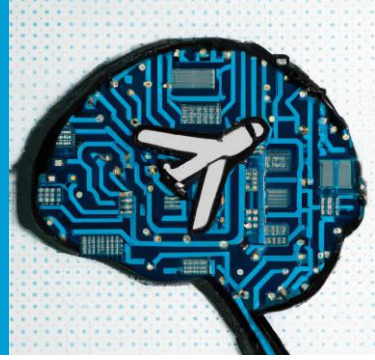
Question 1:



Why do neural networks have problems with modeling the solution to chaotic systems?

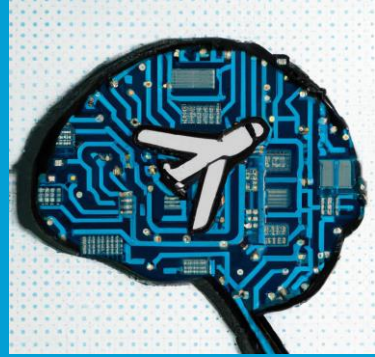
- A. Because neural networks require clean data to function, and chaotic systems are too noisy to provide that.
- B. Because chaotic systems involve random processes, and neural networks are not designed to handle random behavior.
- C. Because the floating point weights in neural networks cannot model imaginary numbers required for chaotic systems.
- D. Because chaotic systems are highly sensitive to initial conditions, making long-term predictions difficult even with small errors.

Question 1:



Why do neural networks have problems with modeling the solution to chaotic systems?

- A. Because neural networks require clean data to function, and chaotic systems are too noisy to provide that.
- B. Because chaotic systems involve random processes, and neural networks are not designed to handle random behavior.
- C. Because the floating point weights in neural networks cannot model imaginary numbers required for chaotic systems.
- D. Because chaotic systems are highly sensitive to initial conditions, making long-term predictions difficult even with small errors.



Questions?